

Language



Автор сайта



Дмитрий Виноградов

Подробнее можно [посмотреть](#) или [почитать](#).

Никакие другие люди не имеют к этому сайту никакого отношения и просто занимаются плагиатом.



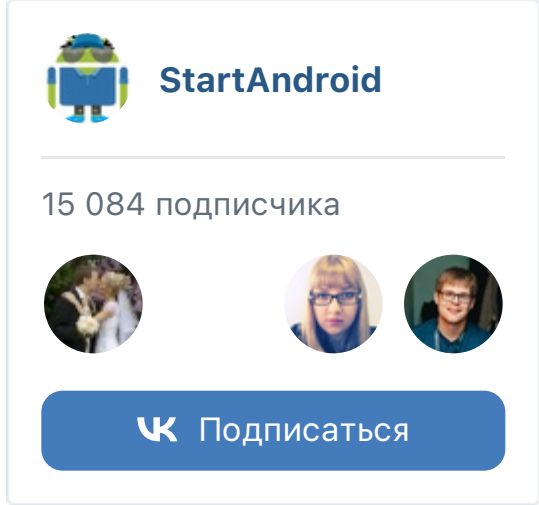
Социальные сети



В [канале](#) я публикую ссылки на интересные и полезные статьи по Android

В [чате](#) можно обсудить вопросы и проблемы, возникающие при разработке

Группа ВКонтакте



Поддержка проекта

Яндекс
410011180491924

WebMoney
R248743991365
Z551306702056

Paypal

Урок 9. Компоненты. Dependencies

21 июня 2021

В этом уроке рассмотрим dependencies связь между компонентами

Кроме сабкомпонентов, даггер предоставляет нам еще один способ взаимодействия между компонентами. В этом случае мы создаем два обычных компонента и в одном из них указываем, что он зависит от другого.

Рассмотрим на том же примере, что и ранее. У нас есть обычный компонент AppComponent:

```
1 @Component(modules = [StorageModule::class, NetworkModule::class])
2 interface AppComponent {
3
4     // ...
5
6 }
```

Мы хотим создать отдельный компонент MainComponent, который будет создавать нам презентер для MainActivity. Компоненту MainComponent для создания презентера понадобятся объекты, которые умеет создавать AppComponent.

В прошлых уроках, чтобы решить эту зависимость, мы MainComponent делали сабкомпонентом для компонента AppComponent. В итоге MainComponent умел все то же, что и AppComponent, а также имел и свой функционал.

Но сейчас мы опишем MainComponent, как обычный компонент (не сабкомпонент):

```
1 @Component(modules = [MainModule::class], dependencies = [AppComponent::class])
2 interface MainComponent {
3
4     fun getMainActivityPresenter(): MainActivityPresenter
5
6 }
```

А в аннотации Component мы используем параметр dependencies, в котором указываем класс AppComponent. Этим мы сообщаем даггеру, что MainComponent должен иметь доступ к объектам компонента AppComponent.

Даггер учитывает это, когда создает класс компонента - DaggerMainComponent. В билдер этого компонента он зашивает необходимость передачи объекта AppComponent.

В итоге создание компонента MainComponent выглядит так:

```
1 val mainComponent = DaggerMainComponent.builder()
2   .AppComponent(appComponent)
3   .build()
```

Мы должны передать ему экземпляр компонента AppComponent. Только так MainComponent сможет получить доступ к объектам AppComponent. Он просто будет просить их у этого компонента.

И вот в этом кроется разница между отношениями компонент-субкомпонент и компонент-компонент.

В прошлых уроках мы разбирали, что когда мы к компоненту добавляем сабкомпонент, то компонент сам создает реализацию этого сабкомпонента. В эту реализацию он передает свои модули. И сабкомпонент сам может в этих модулях найти все, что ему понадобится. Т.е. сабкомпоненту неважно, какие объекты прописаны в интерфейсе компонента. Субкомпонент лезет напрямую в модули, минуя компонент.

А вот в случае пары компонент-компонент связь не такая тесная. Компоненты создаются отдельно друг от друга. Они общаются на уровне интерфейсов и не имеют доступа к модулям. И это большая разница.

В нашем примере компонент MainComponent просит объекты у компонента AppComponent. Но все, что MainComponent знает о AppComponent - это его интерфейс. Т.е. MainComponent может попросить только те объекты, которые явно прописаны get методами в интерфейсе компонента AppComponent. А к модулям AppComponent у него доступа нет.

Т.е. если в интерфейсе AppComponent есть метод:

```
1 fun getDatabaseHelper(): DatabaseHelper
```

то MainComponent сможет получить этот объект, чтобы использовать его для создания своих объектов.

А если такого метода не будет, то компонент MainComponent никак не сможет получить этот объект. Поэтому в AppComponent нам необходимо явно прописать get-методы для объектов, которые нужны будут компоненту MainComponent

В наших примерах MainComponent использует DatabaseHelper и NetworkUtils, чтобы создать презентер:

```
1 @Module
2 class MainModule {
3
4     @Provides
5     fun provideMainActivityPresenter(databaseHelper: DatabaseHelper, networkUtils: NetworkUtils)
6         return MainActivityPresenter(databaseHelper, networkUtils)
7
8
9 }
```

Это означает, что в AppComponent нам необходимо явно прописать get методы для этих объектов

```
1 @Component(modules = [StorageModule::class, NetworkModule::class])
2 interface AppComponent {
3
4     fun getDatabaseHelper(): DatabaseHelper
5     fun getNetworkUtils(): NetworkUtils
6
7 }
```

Иначе MainComponent не сможет их получить.

Нет универсального рецепта, как организовывать компоненты.

Кто-то использует один основной App компонент, а для каждого Activity создает свой сабкомпонент. Кто-то использует один универсальный сабкомпонент для всех Activity. Кто-то вообще не использует сабкомпоненты, а все строит на dependencies.

Официальная документация рекомендует использовать сабкомпоненты, а не dependencies. В случае сабкомпонентов даггер лучше знает, какие объекты требуется создавать, а какие - нет. Это позволит ему сократить количество генерируемого кода.

Назад

Вперёд

Комментарии

RE: Урок 9. Компоненты. Dependencies — Eldar 06.11.2021 01:32

Сабкомпоненты имеют общий граф зависимости с компонентом, тем самым увеличивая время компиляции в больших проектах

Ответить

RE: Сабкомпоненты имеют общий граф зависимости с компонентом, тем самым увеличивая время компиляции в больших проектах — Алексей 24.11.2021 21:27

Да, но время компиляции приложения не так критично, как его конечный размер

Ответить

RE: Урок 9. Компоненты. Dependencies — Андрей 09.10.2022 20:04

В статье на Хабре разработчик из лаборатории Касперского пишет, что субкомпоненты плохи для многомодульного приложения, так как то, что субкомпонент знает обо всех модулях компонента, мешает должным образом изолировать модули друг от друга...
https://habr.com/ru/company/kaspersky/blog/422555/

Ответить

Обновить список комментариев

RSS лента комментариев этой записи

Добавить комментарий

Тема

😄 😁 😂 😃 😅 😆 😇 😈 😊 😋 😌 😍 😎 😏 😐 😑 😒 😓 😔 😕 😖 😗 😘 😙 😚 😛 😜 😝 😞 😟 😠 😡 😢 😣 😤 😥 😦 😧 😨 😩 😪 😫 😬 😭 😮 😯 😰 😱 😲 😳 😴 😵 😶 😷 😸 😹 😺 😻 😼 😽 😾 😿 😽 😿 😺 😻 😼 😽 😾 😿 😽 😿 😺 😻 😼 😽 😾 😿

Отправить

1Comments

